

MUSTAFA SHOUKAT

Senior AI Engineer, Enterprise AI Architecture and Agentic Systems

mustafashoukat.ai@gmail.com +92 309 3609261 linkedin.com/in/mustafashoukat github.com/Mustafa-Shoukat1

Executive Positioning Brief

Prepared for Staff AI Engineer, Teradata (Job ID 220477), 23 September 2026

This brief maps my experience to the requirements of the Staff AI Engineer role. For each area it states what the role asks for, what I have done, and the result.

1. AI platform architecture

The requirement. Design and deliver highly scalable AI products; architect for scalability, reliability and performance.

What I did. I designed ByanRAG 2.2 end to end: FastAPI services, a React admin console with 14 panels, isolated Qdrant collections per organization, a Casbin access model with five role levels checked on every endpoint, and full audit logging. The platform is packaged for on premises and air gapped deployment.

Result. Several organizations share one platform with strict data isolation, and it runs inside regulated networks where cloud services are not an option.

2. Retrieval augmented generation

The requirement. Apply LLMs and retrieval to enterprise data with answers people can trust.

What I did. I built hybrid retrieval that combines dense embeddings with BM25, merges them with Reciprocal Rank Fusion and reranks with a CrossEncoder. It is tuned for Arabic first, bilingual content and streams answers with source citations. Quality is measured with Recall@5, MRR and NLI faithfulness scoring.

Result. Every answer points to its source document, and retrieval quality is tracked as numbers rather than judged by eye.

3. Orchestration and agentic AI

The requirement. Experiment with and deliver agentic AI and large language model systems.

What I did. I lead multi agent workflows in LangGraph and CrewAI with tool calling and multi step planning. One example is a government contracting platform that pulls solicitations from SAM.gov by NAICS code, reads the PDFs through a RAG pipeline, analyses requirements, recommends whether to bid and drafts the proposal, all over a provider agnostic LLM layer. I also build voice and WhatsApp automation with Whisper transcription and human handover.

Result. Analyst work that took days is now automated, and the choice of LLM provider stays open.

4. Distributed systems

The requirement. Strong grasp of distributed system patterns, data consistency and scale.

What I did. I use Celery and Redis for background ingestion, scraping and agent jobs; async PostgreSQL for data; WebSockets and server sent events for streaming; atomic message deduplication and fail closed HMAC verification on webhooks; and swappable vector stores behind one interface.

Result. Work keeps running when a worker fails, retries never create duplicate records, and components can be replaced without rewriting the system.

5. Production readiness

The requirement. Resolve performance bottlenecks, investigate production issues and maintain code quality.

What I did. I run services in Docker Compose behind nginx with TLS, with CI/CD and a suite of more than 600 tests. Monitoring uses Prometheus, Grafana and Loki, with LangSmith tracing and Sentry. Security covers role based access, audit logs, guardrails and prompt injection defenses. I have traced and fixed real bottlenecks such as reranker latency.

Result. Problems surface on dashboards and traces before users report them. The same evaluation discipline took Yehia R1 to first place on the AraGen Leaderboard.

6. Technical leadership

The requirement. Mentor engineers, influence architecture decisions and work independently.

What I did. I own architecture decisions on my platforms and set the team's standards for testing, deployment, code review and model integration. I mentor engineers, have trained more than 150 professionals in data science and AI engineering, work directly with clients in four countries, and contribute to LangChain, RAGAS and Unstructured.

Result. Teams ship to a shared standard, and I can carry a system from first design to production on my own.

Engineering foundations

Languages. Python as my primary language, with Java, TypeScript and SQL. **APIs and data.** RESTful APIs in FastAPI; data modeling in PostgreSQL, MongoDB, Redis and pgvector with SQLAlchemy and Alembic. **Cloud and containers.** AWS, GCP and Azure, with Docker in production. **AI productivity tools.** I use AI coding assistants such as GitHub Copilot every day.

My container orchestration so far has been Docker Compose rather than Kubernetes, and my messaging has been Celery and Redis rather than Kafka. The distributed patterns are the same, and I am ready to work in both from the start.

Summary

I bring proven, hands on ownership of the six areas this role centers on: platform architecture, RAG, agentic orchestration, distributed systems, production readiness and technical leadership, backed by systems that run in production today.